

Tutorial 14: Running the League

A practical local-server Screeps module

[docs/tutorials/14-running-the-league.md](#)

Episode 14: Open the Gates

"There's one role left," your collaborator says, "and it's not a creep. It's you. Somebody has to run the league the next architect drops into – congratulations, you've been promoted to management."

Thirteen episodes ago you were a single hardcoded creep bouncing between one source and one spawn. You're being asked to build the thing that trained you. That's either the most startup thing that's ever happened to you, or the most startup thing that's about to.

Goal

Run one complete tournament cycle end to end, exercising every part of the operation before real contestants ever touch it:

- Set up a submissions structure and lock entries at a deadline
- Choose and justify a match format
- Pass every submission through an anti-cheat checklist
- Generate a bracket
- Run and record at least one match
- Score it and publish the result

Prerequisites

Tutorial 13 is complete:

- `scripts/sparring-loop.sh` and the sparring-ground mod are working – you've run at least one multi-wave test against your own colony.
- `league/README.md` and `league/scoring-rubric.md` exist in this repo – read both before continuing.

Step 1: Open Submissions

Every contestant gets a folder under `league/submissions/<callsign>/`, containing their bot's source and a `SUBMISSION.md`:

```
Submission: <callsign>
```

```
Commit: <git commit hash>
```

```
Submitted: <timestamp>
```

```
Strategy: <one line – what is this bot actually trying to do?>
```

For your first internal run, submit your own Episode 12/13 defender logic under a made-up callsign. Running the full pipeline against your own work first is how you find its gaps before a real contestant does.

Step 2: Lock Submissions at the Deadline

Once a submission is in, freeze it. Tag the exact commit:

```
git tag submission-<callsign>-<date>
```

No changes to that folder count after the tag is cut. This isn't bureaucracy – it's the only thing that makes "here's who won" a claim you can actually defend later. A submission that changed after the deadline isn't the submission that was judged.

Step 3: Choose a Match Format

Two real options, from what this season already built:

- **Sparring benchmark** (Episode 13): every contestant's bot runs on its own colony, on its own schedule, and gets scored by how it holds up against the same standardized `scripts/sparring-loop.sh` wave sequence – same room-setup expectations, same wave count, same delay. This sidesteps the hardest part of a private-server tournament entirely: two contestants never need to be online or claimed into the same room at the same time. It's fully offline, and every result is reproducible by rerunning the identical wave sequence.
- **World colony vs. colony**: two contestants' bots actually coexist and fight in the same room or set of rooms. More faithful to real PvP, but genuinely harder to operate – there's no built-in "match over" signal, and getting two different codebases running as two different local accounts in the same private server takes real setup this season didn't build out.

For a first tournament, use the sparring benchmark. It's the format this repo can already run end to end, asynchronously, with nothing beyond what Episodes 7, 12, and 13 already shipped.

Step 4: Run Every Submission Through the Anti-Cheat Checklist

Before any submission is eligible to be matched:

- The commit hash in `SUBMISSION.md` matches an actual tagged commit – no untagged, unverifiable code.
- The code doesn't attempt anything the sandboxed runtime wouldn't allow anyway (network calls, filesystem access) – if it's in the submission, that's a signal to look closer at intent even if the sandbox would block it.
- The code doesn't rely on a scenario-specific exploit that isn't part of the published rules for that match format.
- The submission folder wasn't modified after its tag was cut – `git log --oneline league/submissions/<callsign>` should show nothing after the tagged commit.

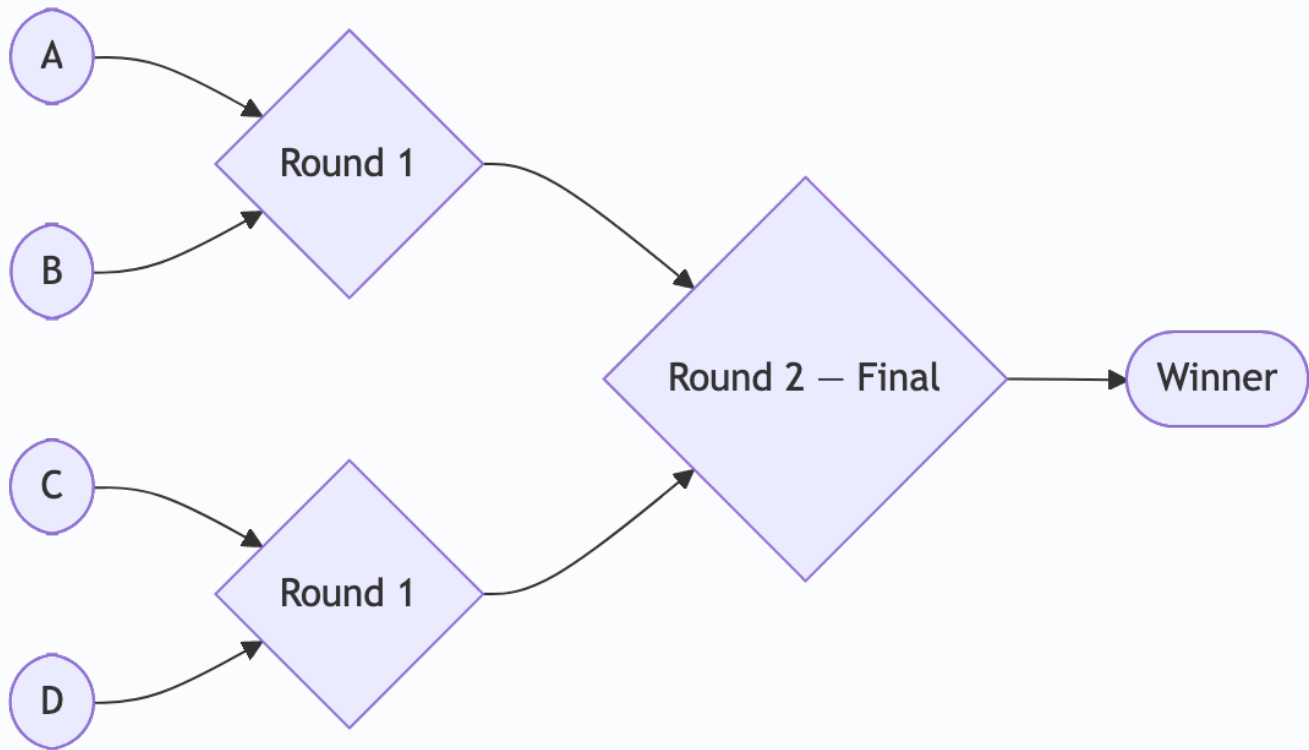
A submission that fails any item is disqualified from the match it was entered into, not quietly patched and re-run. Record the disqualification the same way you'd record a result.

Step 5: Generate a Bracket

For a small first run, single elimination is enough. With contestants `A`, `B`, `C`, `D`:

```
Round 1: A vs B, C vs D
Round 2: winner(A/B) vs winner(C/D)
```

Seed order matters for fairness at scale, but for your first internal run, a simple shuffle is fine – document how you seeded it (`league/results/` is where that documentation lives) so the process is repeatable and inspectable later, not just remembered.



Step 6: Run and Record a Match

Deploy the locked submission's code to its own colony on this server, then run the exact same benchmark against it:

```
./scripts/sparring-loop.sh <contestant-room-name> 5 10
```

Use identical arguments for every contestant in a given round – that consistency is what makes the results comparable at all. While it runs, capture:

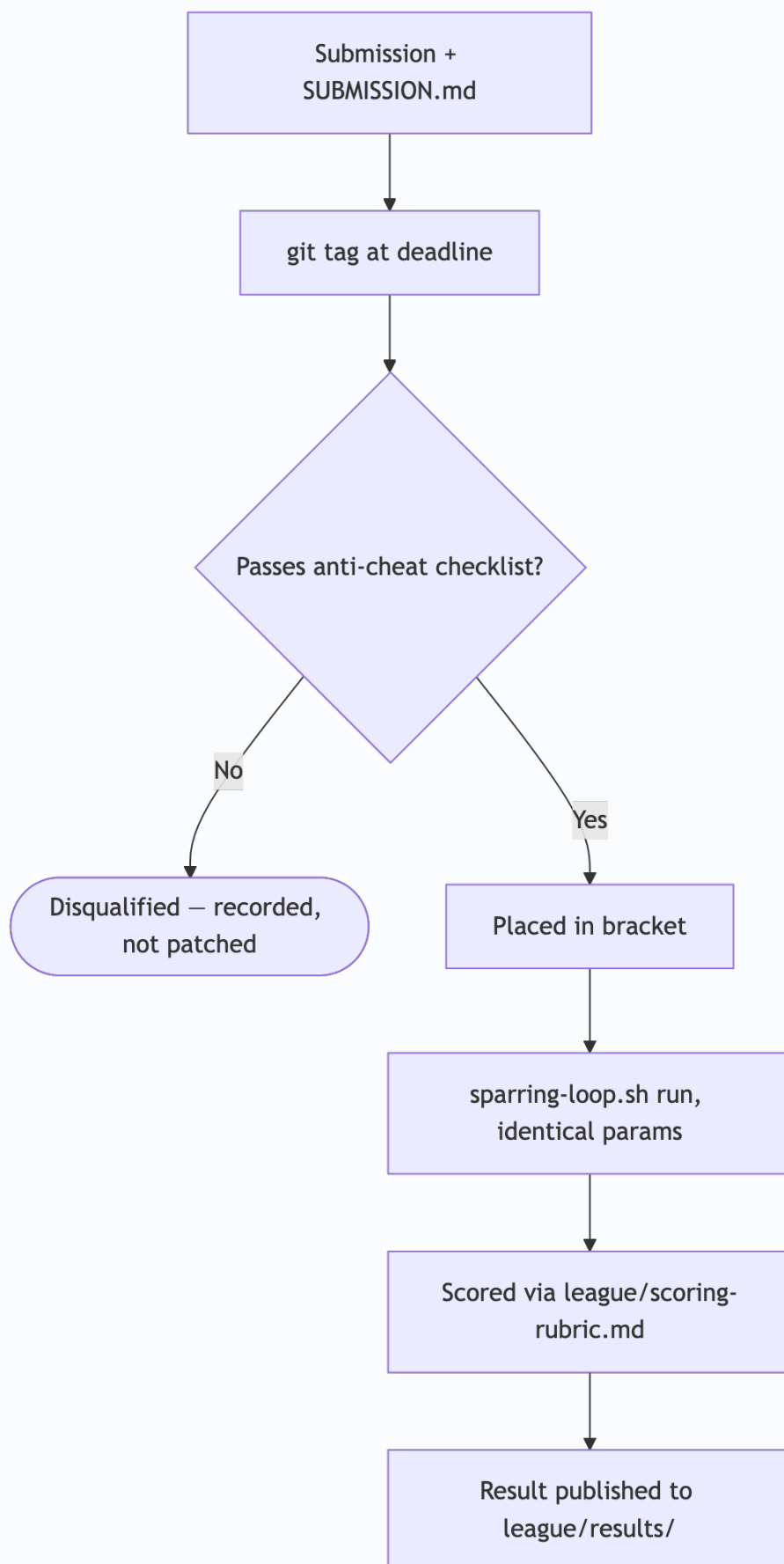
- The contestant's callsign and commit hash.
- How many waves the colony's defense held before failing (or that it held all of them).
- What specifically failed first, if it did – same observation you practiced making in Episode 13.

Step 7: Score and Publish

Fill out the result template from `league/scoring-rubric.md` and save it as `league/results/<match-id>.md`. Use the Match Result table for the win/loss score; use the Judged Categories table only if this is a showcase or cohort format where code quality is part of what's being taught, not just who won.

Checkpoint: `league/results/` has at least one completed, filled-out match file, referencing a real tagged commit hash for each contestant.

The full pipeline, start to finish:



 **Screenshot placeholder:** A contestant's colony mid-benchmark, tower firing on the current wave, with the terminal's wave log visible in the same frame – the "match in progress" shot for a league recap.

Troubleshooting

If you can't tell whether a contestant "won," that's a sign the benchmark's success condition wasn't clearly defined before submissions opened – decide up front whether waves-survived, time-to-failure, or a judged rubric score is the actual metric, and publish that rule before the next round runs.

If a submission's commit hash doesn't match what's in its folder, don't guess which version is "the real one." Disqualify and ask the contestant to resubmit through the proper channel before the next deadline.

If two contestants' bots behave identically, check whether one is a copy of the other before assuming it's a coincidence – this is exactly what the anti-cheat checklist in Step 4 exists to catch.

Completion Criteria

You are done when:

- At least one submission exists under `league/submissions/` with a valid, tagged commit.
- It passed every item on the anti-cheat checklist.
- At least one match has been run, scored, and recorded in `league/results/`.
- You could hand this process to someone else and they could run the next round without you explaining it verbally.

Learning Notes

After completing the tutorial, write down:

- What broke the first time you tried to run this end to end that you didn't expect?
- Which step in this pipeline took the longest, and is that step something you'd automate before running a real cohort through it?
- What's the smallest rule you'd add to the anti-cheat checklist after actually trying to break your own process?
- Look back at the very first entry in `docs/journal/`. What's different about how you approach a bug now versus then?

Season Recap

Fourteen episodes ago, AutoNate was a callsign on a drop pod hull and a blank room with a blinking cursor. No deck, no runway, no cofounder – just a spawn and a bad idea that turned out not to be bad. The colony that exists now runs itself: sources assigned without collision, roles dispatched without hardcoded names, a controller climbed on purpose, roads and containers built ahead of need, mining and hauling split for throughput, a tower and ramparts that hold a line, a second room reserved and feeding the first, CPU treated as a budget instead of an afterthought, a debugging practice worth writing down, scouting that remembers what it saw, a defender that knows the difference between a threat and an empty room, a fast lab for testing combat before it's real, and now – a league other architects can drop into.

"The Foundry didn't train you to win," your collaborator says. "It trained you to build something that keeps running after you stop watching it. You just did that twice – once with a colony, once with a league. Most founders take a decade to learn that. You did it in fourteen episodes, and you didn't even need a Series A."

See `docs/roadmap.md` for the full season, start to finish.